

Cyclone Core

Deterministic XiL scenario-testing platform for ADAS domain controllers

Inject determinism. Output evidence chains.

Deterministic fault injection × statistical verdicts × auditable evidence chains — built for functional safety & SOTIF validation

Are these three problems holding back your integration testing?

- 01 Fault injection is manual.** Pulling cables, patching code, writing one-off scripts — the same fault can never be reproduced precisely a second time, and regression conclusions cannot be aligned.
- 02 Degradation-chain timing cannot be quantified.** Sensor frame drops, delays and data corruption propagate along the chain; there is no quantifiable, assertable way to verify at which millisecond the system degrades, or whether it meets the safety time window.
- 03 Audit evidence relies on screenshots.** ISO 26262 / SOTIF audits demand a traceable, re-checkable evidence chain — not a Word document stitched together from screenshots.



Four Core Capabilities CAPABILITIES

Deterministic Fault Injection FaultPipe

KEY DIFFERENTIATOR

- **Five fault primitives:** Drop (frame drop), Delay, CorruptField (field corruption), Freeze and CorruptCrc (CRC error) cover typical failure modes at the bus and signal layers.
- **Tick-precise triggering:** declare trigger points by time window, frame count or conditional expression — the same fault is guaranteed at tick N, with reproduction granularity down to the scheduling cycle.
- **YAML declarative configuration:** fault scenarios as code — reviewable, versionable, diffable. No more one-off hand-written scripts.

Deterministic Execution Kernel Time Master

- **Dual-mode clock:** real-time mode connects to hardware-in-the-loop; virtual-time mode accelerates replay on ordinary servers — the same test case yields identical conclusions in both modes.
- **1 ms single-threaded deterministic scheduling:** identical inputs produce bitwise-identical outputs; test conclusions do not depend on the machine or the moment of execution.

Statistical Assertion Verdicts Verdict Engine

- **Assertions as latching state machines:** Pending → Satisfied / Violated / TimedOut — once latched, a verdict never reverts; the verdict process is explainable and auditable.
- **Built-in statistical verdicts:** Wilson confidence intervals and SPRT sequential testing turn "ran it a few times, feels fine" into "passed at 95% confidence".

Auditable Evidence Chain Evidence Chain

KEY DIFFERENTIATOR

- **Four artifacts produced automatically:** case hash (unique scenario identifier) + SHA-256 log digest + failure-scene snapshot + JUnit XML report.
- **Designed for audits:** every test conclusion traces back to a specific case version and its raw data; supports third-party re-verification and plugs directly into CI regression dashboards.

Compared with Your Current Workflow WHY CYCLONE CORE

Table 1 — Fault injection & verdict approaches compared

Dimension	Manual scripts	CANoe manual operation	Cyclone Core
Fault reproducibility	Low, depends on individual skill	Medium; operation sequences hard to codify	Bitwise reproducible; configuration as an asset
Injection precision	Second-level, luck-dependent	Depends on human click timing	Tick-level (1 ms scheduling cycle)
Verdict method	Eyeballing logs	Manual panel observation	Latching state machine + statistical confidence
Evidence output	Screenshots / hand-written docs	Requires manual re-organization	Hash + snapshot + JUnit, produced automatically
CI integration	Difficult	Requires extra licenses and modification	Native CLI, ready to plug in

Positioning note: Cyclone Core does not replace your existing CANoe / dSPACE / programmable power-supply setup — it sits on top as a deterministic fault-injection and evidence layer, complementing the output of your existing toolchain.

Interfaces & Integration INTERFACES

SocketCAN **Supported**UDP **Supported**YAML Test DSL **Supported**JUnit XML / CLI **Supported**DDS **Prototype**SOME/IP **Planned**UDS Diagnostics **Planned**ASAM XIL API **Planned**

Note: interfaces marked "Prototype / Planned" have not yet reached delivery maturity; during the POC phase we can evaluate priorities against your bus environment.

Case Study: AEB Perception Degradation-Chain Validation DEMO SCENARIO

An AEB function needs to verify: after front-camera frame-drop faults are injected, within how many milliseconds the perception fusion completes its degradation, and whether AEB exits or degrades braking within the safety time window. The table below shows a real run from Cyclone Core's lightweight demo environment (simulation replay + Python verdict pipeline):

Table 2 — Baseline vs fault-injection verdict flip (demo data, from a simulation-replay environment)

Scenario	Near-range recall	Mid-range recall	Far-range recall	False-alarm rate	System verdict
Baseline (no fault)	0.9924	0.7040	0.3453	0.0000	PASS
Frame-drop injection	0.8724	—	0.0000	0.1033	FAIL (verdict flip)

Demo environment limitations: the data above comes from a lightweight demo pipeline and illustrates the full closed loop of "fault injection → metric degradation → verdict flip → evidence output"; the formal POC will be executed with the same methodology on your real bus and ECU environment.

Engagement Models & Reference Pricing COMMERCIAL

Proof of Concept (POC)

RMB 100K–300K

~3-month engagement, 50% upfront; delivers one complete degradation-chain validation with evidence chain in your real scenario; the POC fee is fully deductible from the formal purchase.

Annual Subscription

RMB 80K–150K / seat / year

Includes all feature updates and remote support; ideal for test teams rolling out per seat, deeply integrated with the CI pipeline.

Perpetual License

RMB 250K–400K

18% annual maintenance included (optional); test-bench runtime licenses also available at RMB 20K–30K / node / year. Channel partnerships supported (65–70% channel price).

The above is reference pricing; final terms are subject to the formal quotation and technical agreement. Evidence-chain formats can be customized to functional-safety audit requirements.

Our Commitment COMMITMENT

Enhance, not replace. Cyclone Core is positioned as a deterministic fault-injection and evidence layer on top of your existing CANoe / dSPACE / in-house script setup, protecting your existing toolchain investment.

Re-checkable conclusions. Every PASS / FAIL can be traced back to a case hash and raw data; your team and third-party organizations are welcome to independently re-verify the demo results.

Private deployment. Pure-software delivery with offline intranet deployment supported; test data never leaves your network boundary.

Business contact

Tonny Cao

Email

caojian258@icloud.com

Phone / WeChat

WeChat: jiancao258

Technical demo

[Book a 30-min online demo]